Лабораторная работа №3

Компьютерный практикум по статистическому анализу данных

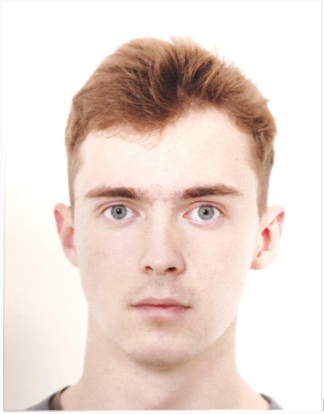
Лисенков Е.Р.

Российский университет дружбы народов, Москва, Россия

Информация

Докладчик

- Лисенков Егор Романович
- студент НФИбд-02-23
- Российский университет дружбы народов
- 1132232881@rudn.ru
- https://github.com/erlisenkov



Ц

Вводная часть

Актуальность

Циклы, условные выражения и функции составляют основу управляющих структур любого языка программирования, и их освоение в Julia необходимо для решения задач линейной алгебры и работы с матрицами.

Объект и предмет исследования

Объектом выступают управляющие структуры языка Julia, а предметом — применение циклов, условных выражений, функций и функций высшего порядка для решения вычислительных задач.

Цели и задачи

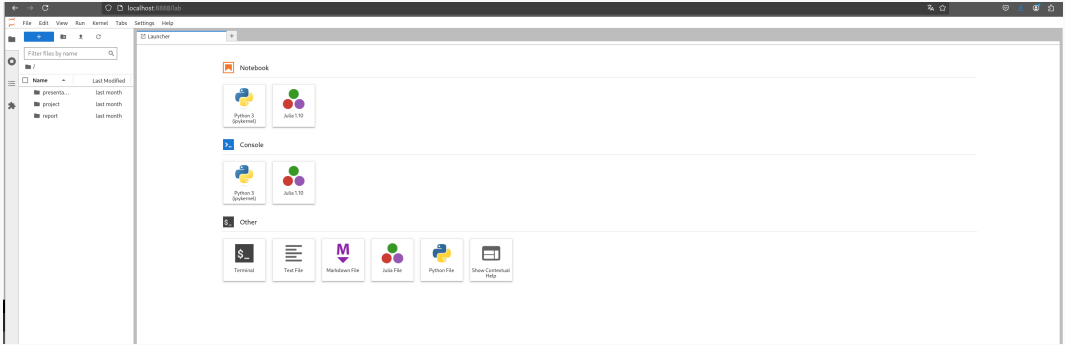
Целью работы является освоение циклов, условных выражений и способов определения функций, а задачами — воспроизведение примеров методических указаний и выполнение одиннадцати заданий самостоятельной работы.

Материалы и методы

Работа выполнена в операционной системе Fedora с использованием языка Julia версии 1.10.3, интерактивной среды Jupyter Lab и системы контроля версий Git.

Циклы while и for

Примеры циклов while и for над числами и строковыми массивами подтвердили эквивалентность подходов и выразительность цикла for при переборе коллекций.



Страница запуска Jupyter Lab

Работа выполнена в среде Jupyter Lab с выбором ядра Julia 1.10 из доступных ядер Python 3 и Julia.



Создание матриц и условное выражение FizzBuzz

Матрица с элементами, равными сумме индексов, построена тремя способами: вложенными циклами, комбинированным циклом и генератором, а условное выражение FizzBuzz воспроизведено для числа 15.



Анонимные функции, сортировка и функции высшего порядка

Соглашение о восклицательном знаке проверено сравнением sort и sort!, а различие матричной и поэлементной операций продемонстрировано функцией map и вызовами f(A) и f.(A).



Три способа определения функций и тернарный оператор

Функции определены через ключевые слова function и end, в одну строку и как анонимные, а эквивалентность тернарного оператора условному выражению подтверждена значением 10.



Задание 1: циклы, словарь и массив квадратов

Квадраты чисел от 1 до 100 получены тремя способами: выводом в цикле, накоплением в

```
# Вывод чисел и их квадратов
for i in 1:100
    println("$i^2 = $(i^2)")
end

# Словарь squares
squares = Dict{Int64, Int64}()
for i in 1:100
    squares[i] = i^2
end
println("Словарь squares создан, размер: ", length(squares))

# Массив squares_arr
squares_arr = [i^2 for i in 1:100]
println("Массив squares_arr: ", squares_arr[1:10], "...")
```

```
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81
10^2 = 100
11^2 = 121
12^2 = 144
13^2 = 169
14^2 = 196
15^2 = 225
16^2 = 256
17^2 = 289
18^2 = 324
19^2 = 361
20^2 = 400
21^2 = 441
22^2 = 484
23^2 = 529
24^2 = 576
25^2 = 625
26^2 = 676
27^2 = 729
28^2 = 784
29^2 = 841
30^2 = 900
31^2 = 961
32^2 = 1024
33^2 = 1089
34^2 = 1156
35^2 = 1225
36^2 = 1296
37^2 = 1369
38^2 = 1444
39^2 = 1521
40^2 = 1600
41^2 = 1681
```

Задание 2: условный оператор и тернарный оператор

Для проверочного значения 7 обе формы ветвления вывели строку «нечётное», что подтвердило эквивалентность конструкции if-else и тернарного оператора.

```
# Обычный условный оператор
number = 7
if number % 2 == 0
    println(number)
else
    println("нечётное")
end

# Тернарный оператор
result = (number % 2 == 0) ? number : "нечётное"
println("Результат через тернарный оператор: ", result)
```

```
нечётное
Результат через тернарный оператор: нечётное
```

Задание 3: функция add_one

Функция, добавляющая единицу к своему входу, определена через ключевые слова function и end и вернула значения 6 и 11 для контрольных вызовов.

[19]:
```
function add_one(x)
    return x + 1
end

println("add_one(5) = ", add_one(5))
println("add_one(10) = ", add_one(10))
```

```
add_one(5) = 6
add_one(10) = 11
```

[ ]:

Точечный синтаксис broadcast увеличил каждый элемент матрицы 5x5 на единицу без изменения её структуры.

```
add_one(10) = 11
```

[20]:
```
# Создаём матрицу 5x5, где каждый элемент увеличивается на 1
A = [i + j for i in 1:5, j in 1:5]
println("Исходная матрица A:")
println(A)

# Используем broadcast для увеличения каждого элемента на 1
B = A .+ 1
println("Матрица после увеличения на 1:")
println(B)
```

```
Исходная матрица A:
[2 3 4 5 6; 3 4 5 6 7; 4 5 6 7 8; 5 6 7 8 9; 6 7 8 9 10]
Матрица после увеличения на 1:
[3 4 5 6 7; 4 5 6 7 8; 5 6 7 8 9; 6 7 8 9 10; 7 8 9 10 11]
```

Задание 5: куб матрицы и замена столбца

Куб заданной матрицы оказался нулевым вследствие нильпотентности, а замена третьего столбца на сумму второго и третьего дала ожидаемую матрицу.

```
[3 4 5 6 7; 4 5 6 7 8; 5 6 7 8 9; 6 7 8 9 10; 7 8 9 10 11]
```

```
]:
A = [1 1 3; 5 2 6; -2 -1 -3]
println("Исходная матрица A:")
println(A)

# Находим A^3
A3 = A^3
println("A^3 =")
println(A3)

# Заменяем третий столбец на сумму второго и третьего
A[:, 3] = A[:, 2] + A[:, 3]
println("Матрица A после замены третьего столбца:")
println(A)
```

```
Исходная матрица A:
[1 1 3; 5 2 6; -2 -1 -3]
A^3 =
```

Задание 6: матрица B и произведение C = B^T * B

Матрица 15x3 с постоянными столбцами заполнена в цикле, и произведение с транспонированной матрицей дало матрицу с элементами 1500 и минус 1500.

```
[1 1 4; 5 2 6; -2 -1 -4]
```

[*]:
```
# Создаём матрицу B 15x3
B = zeros(15, 3)
for i in 1:15
    B[i, 1] = 10
    B[i, 2] = -10
    B[i, 3] = 10
end
println("Матрица B (15x3):")
println(B)

# Вычисляем C = B^T * B
C = transpose(B) * B
println("Матрица C = B^T * B:")
println(C)
```

Задание 7: построение матриц Z1-Z4

Четыре матрицы 6x6 с заданными закономерностями расположения единиц построены из нулевой матрицы вложенными циклами с условиями на чётность сумм индексов.

```
[31]: # Матрица Z1
      Z1 = zeros(Int64, 6, 6)
      for i in 1:6
          for j in 1:6
              if (i == 1 && j == 2) || (i == 2 && j == 1) || (i == 2 && j == 3) ||
                 (i == 3 && j == 2) || (i == 3 && j == 4) || (i == 4 && j == 3) ||
                 (i == 4 && j == 5) || (i == 5 && j == 4) || (i == 5 && j == 6) ||
                 (i == 6 && j == 5)
                  Z1[i, j] = 1
              end
          end
      end
      println("Z1 =")
      println(Z1)

      # Матрица Z2
      Z2 = zeros(Int64, 6, 6)
      for i in 1:6
          for j in 1:6
              if (i + j) % 2 == 0 && !(i == 1 && j == 1) && !(i == 6 && j == 6)
                  Z2[i, j] = 1
              end
          end
      end
      println("Z2 =")
      println(Z2)

      # Матрица Z3
      Z3 = zeros(Int64, 6, 6)
      for i in 1:6
          for j in 1:6
              if (i + j) % 2 == 1
                  Z3[i, j] = 1
              end
          end
      end
      println("Z3 =")
      println(Z3)

      # Матрица Z4
      Z4 = zeros(Int64, 6, 6)
      for i in 1:6
          for j in 1:6
              if (i + j) % 2 == 1 || (i == 1 && j == 1) || (i == 6 && j == 6)
                  Z4[i, j] = 1
              end
          end
      end
      println("Z4 =")
      println(Z4)
```

Задание 8: функция outer и матрицы A1-A5

Реализована собственная функция outer, аналогичная функции языка R, и с её помощью построены пять матриц заданной структуры, включая циклические и степенные.

```
# Функция outer
function outer(x, y, operation)
    m = length(x)
    n = length(y)
    result = zeros(m, n)
    for i in 1:m
        for j in 1:n
            result[i, j] = operation(x[i], y[j])
        end
    end
    return result
end

# A1: сумма индексов
x1 = 0:4
y1 = 0:4
A1 = outer(x1, y1, +)
println("A1 =")
println(A1)

# A2: степени
x2 = 0:4
y2 = 0:4
A2 = outer(x2, y2, (a, b) -> a^b)
println("A2 =")
println(A2)

# A3: остаток от деления
x3 = 0:4
y3 = 0:4
A3 = outer(x3, y3, (a, b) -> (a + b) % 5)
println("A3 =")
println(A3)

# A4: сдвиг по модулю 10
x4 = 0:9
y4 = 0:9
A4 = outer(x4, y4, (a, b) -> (a + b) % 10)
println("A4 =")
println(A4)

# A5: обратный сдвиг
x5 = 0:8
y5 = 0:8
A5 = outer(x5, y5, (a, b) -> (8 - a + b) % 9)
println("A5 =")
println(A5)
```

```
A1 =
[0.0 1.0 2.0 3.0 4.0; 1.0 2.0 3.0 4.0 5.0; 2.0 3.0 4.0 5.0 6.0; 3.0 4.0 5.0 6.0 7.0; 4.0 5.0 6.0 7.0 8.0]
A2 =
[1.0 0.0 0.0 0.0 0.0; 1.0 1.0 1.0 1.0 1.0; 1.0 2.0 4.0 8.0 16.0; 1.0 3.0 9.0 27.0 81.0; 1.0 4.0 16.0 64.0 256.0]
A3 =
[0.0 1.0 2.0 3.0 4.0; 1.0 2.0 3.0 4.0 0.0; 2.0 3.0 4.0 0.0 1.0; 3.0 4.0 0.0 1.0 2.0; 4.0 0.0 1.0 2.0 3.0]
A4 =
[0.0 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0; 1.0 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 0.0; 2.0 3.0 4.0 5.0 6.0 7.0 8.0 9.0 0.0 1.0; 3.0 4.0 5.0 6.0 7.0 8.0 9.0 0.0 1.0 2.0; 4.0 5.0 6.0 7.0 8.0 9.0 0.0 1.0 2.0 3.0; 5.0 6.0 7.0 8.0 9.0 0.0 1.0 2.0 3.0 4.0; 6.0 7.0 8.0 9.0 0.0
```

Задание 9: решение системы линейных уравнений

Система пяти уравнений с тёплицевой матрицей коэффициентов решена оператором обратного слэша с решением [-2, 3, 5, 2, -4], подтверждённым контрольной подстановкой.

5]:
```
# Матрица коэффициентов
A = [1 2 3 4 5;
     2 1 2 3 4;
     3 2 1 2 3;
     4 3 2 1 2;
     5 4 3 2 1]

# Вектор правых частей
y = [7, -1, -3, 5, 17]

# Решение системы Ax = y
x = A \ y
println("Решение системы:")
println("x = ", x)

# Проверка
println("Проверка A*x = ", A * x)
```

```
Решение системы:
x = [-2.0000000000000036, 3.0000000000000058, 4.999999999999998, 1.9999999999999991, -3.999999999999999]
Проверка A*x = [7.0000000000000036, -1.0000000000000036, -3.0, 5.000000000000002, 17.0]
```

Задание 10: статистический анализ случайной матрицы

Для случайной матрицы 6x10 подсчитаны числа элементов выше порога в строках, найдены строки с ровно двумя вхождениями заданного значения и проверены пары столбцов с суммой выше порога.

```
using Random
Random.seed!(42)

# Создаём матрицу M 6x10
M = rand(1:10, 6, 10)
println("Матрица M:")
println(M)

N = 4
# Число элементов > N в каждой строке
counts = [sum(M[i, :] .> N) for i in 1:6]
println("Число элементов > $N в каждой строке: ", counts)

M_val = 7
# Строки, где M встречается ровно 2 раза
rows_with_M = [i for i in 1:6 if sum(M[i, :] .== M_val) == 2]
println("Строки, где $M_val встречается ровно 2 раза: ", rows_with_M)

K = 75
# Пары столбцов с суммой > K
pairs = []
for i in 1:10
    for j in i+1:10
        if sum(M[:, i]) + sum(M[:, j]) > K
            push!(pairs, (i, j))
        end
    end
end
println("Пары столбцов с суммой > $K: ", pairs)
```

```
Матрица M:
```

Задание 11: вычисление двойных сумм

Двойные суммы с разделимым и неразделимым знаменателем вычислены генераторами с двумя переменными перебора и равны 639215.28 и 89912.02.

[27]:
```
# Первая сумма
sum1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
println("Первая сумма = ", sum1)

# Вторая сумма
sum2 = sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
println("Вторая сумма = ", sum2)
```

```
Первая сумма = 639215.2833333338
Вторая сумма = 89912.02146097131
```

Вывод

В ходе лабораторной работы освоены управляющие структуры языка Julia: циклы while и for, условные выражения с ключевыми словами и тернарные операторы, три способа определения функций и функции высшего порядка с точечным синтаксисом. Выполнены все одиннадцать заданий самостоятельной работы, включая матричные операции, решение системы линейных уравнений обобщаемым методом и вычисление двойных сумм, что подтвердило эффективность и обобщаемость применённых подходов. Результаты зафиксированы в системе контроля версий Git.